



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Departamento de Informática



MANUAL TÉCNICO

Autor/es: Alberto García Izquierdo
Curso Académico: DAW 2ºA 25/26

ÍNDICE

1. Introducción.....	3
2. Arquitectura de la aplicación.....	4
2.1 Frontend.....	4
2.1.2 Tecnologías usadas.....	4
2.1.2 Justificación de la elección:.....	5
2.1.3 Entorno de desarrollo.....	5
2.2 Backend.....	5
2.2.1 Tecnologías usadas.....	5
2.2.2 Justificación.....	6
2.2.3 Entorno de desarrollo.....	6
3. Documentación Técnica.....	6
3.1 Análisis.....	6
3.1.1 Roles principales.....	6
3.1.2 Requisitos funcionales.....	7
3.1.3 Requisitos no funcionales.....	9
3.2 Desarrollo.....	11
3.2.1 Base de Datos.....	11
3.2.2 Justificación del diseño.....	14
3.2.3 Diagrama de casos de uso.....	14
3.3 Pruebas realizadas.....	18
4. Proceso de despliegue.....	19
4.1 Software necesario.....	19
4.2 Pasos para desplegar la aplicación.....	19
5. Propuestas de mejoras.....	21
6. Bibliografía.....	21

1. Introducción

FITBOX es una aplicación web desarrollada como proyecto de fin de grado para el ciclo de Técnico Superior en Desarrollo de Aplicaciones Web. Su finalidad principal es ofrecer a los centros deportivos de alto rendimiento una plataforma de software unificada y centralizada que resuelva de forma definitiva la dispersión informática habitual del sector, donde se suelen emplear sistemas totalmente inconexos para la facturación, la reserva de actividades dirigidas y el control técnico del material.

La aplicación adopta una arquitectura de desarrollo front-only: toda la lógica de renderizado, protección de flujos, validación de datos en tiempo real e interacciones reside en el navegador del cliente (React + TypeScript + Vite + Tailwind CSS v4). Por su parte, la persistencia de datos, la gestión de sesiones seguras y el almacenamiento de recursos multimedia se delegan en Supabase, que actúa como un entorno consolidado de Backend-as-a-Service (BaaS). Esta decisión simplifica el despliegue del proyecto al prescindir de servidores de aplicaciones tradicionales e incrementa la seguridad al concentrar las reglas de integridad y persistencia en la nube.

A nivel operativo y funcional, la plataforma ofrece las siguientes capacidades clave:

- Registro y validación de usuarios mediante mecanismos criptográficos administrados por Supabase Auth.
- Control de acceso basado en tres perfiles independientes (Administrador, Monitor y Socio) acotados mediante barreras de enrutamiento en cliente.
- Control dinámico de membresías mediante un bloqueo automático del ecosistema para socios con cuotas inactivas o pendientes de cobro.
- Pasarela financiera simulada con validación estricta de formatos de tarjetas de crédito.
- Estructura CRUD completa para supervisar el inventario de maquinaria pesada y reportar incidencias técnicas en tiempo real.
- Motor de asignación de clases colectivas sujeto a reglas estrictas de aforo máximo y control de asistencia presencial.
- Mecanismo de gamificación transaccional (Experiencia y Niveles) automatizado

mediante disparadores en el motor relacional de la base de datos.

2. Arquitectura de la aplicación

La arquitectura de FITBOX se asienta sobre el patrón de una Single Page Application (SPA) modular desarrollada con React y TypeScript, desacoplada mediante el Patrón Repositorio. Esto garantiza que los componentes visuales nunca realicen peticiones directas o manejen URLs del servidor, aislando las consultas a la base de datos en métodos limpios, reutilizables y altamente escalables.

2.1 Frontend

2.1.2 Tecnologías usadas

- Lenguaje principal: TypeScript (TSX) para añadir tipado estricto al proyecto, eliminando fallos en la manipulación de datos en tiempo de ejecución.
- React v19: Núcleo de la SPA basado en componentes funcionales estructurados y hooks nativos de sincronización de efectos.
- Vite: Entorno de desarrollo ágil y empaquetador del código fuente (bundler) con recarga en caliente optimizada.
- React Router DOM: Gestor del enrutamiento de la aplicación, encargado de aplicar guards lógicos para interceptar accesos no autorizados en base al rol.
- Tailwind CSS v4: Framework de utilidades de hojas de estilo encargado del maquetado visual, soporte nativo de modo oscuro (dark mode) y adaptabilidad de resoluciones bajo enfoque Mobile-First.
- Zustand: Almacén de estado global ligero y persistente encargado de mantener la sesión activa del usuario sin incurrir en sobrecargas de rendimiento o re-renderizados innecesarios.
- React Hook Form: Librería de procesamiento y envío eficiente de formularios.
- Recharts: Librería de renderizado de gráficas interactivas basadas en SVG para la sección analítica de dirección.
- Lucide React: Colección de iconos vectoriales limpios.

2.1.2 Justificación de la elección:

- React 19 y Vite ofrecen el ecosistema de desarrollo más veloz de la actualidad, reduciendo el código repetitivo (boilerplate) y optimizando el empaquetado final para producción.
- TypeScript actúa como un escudo de calidad, impidiendo el paso de datos corruptos o perfiles incompletos hacia las funciones operativas de la interfaz.
- Tailwind v4 agrupa los estilos en propiedades de línea utilitarias, aligerando el peso de transferencia de la red y garantizando que el diseño responsive funcione por igual en cualquier dispositivo móvil o de escritorio.
- Zustand se prefiere sobre Redux debido a que ofrece un almacén de estados centralizado mucho más limpio, directo y compatible con la persistencia automática de tokens de sesión tras recargar la ventana del navegador.

2.1.3 Entorno de desarrollo

- Sistema Operativo: Windows 11.
- IDE: Visual Studio Code optimizado con extensiones de tipado de TypeScript, autocompletado de Tailwind utilitario y depuradores integrados.
- Gestor de Dependencias: npm (Node Package Manager).
- Control de versiones: Git local integrado con repositorios remotos privados en GitHub para blindar la confidencialidad del proyecto.

2.2 Backend

2.2.1 Tecnologías usadas

- Supabase (BaaS): Plataforma unificada en la nube que encapsula y despliega los siguientes servicios críticos de backend:
- PostgreSQL: Motor relacional estructurado encargado de la consistencia de los datos, restricciones transaccionales y almacenamiento analítico.
- Supabase Auth: Módulo serverless de identidad encargado del control de inicios de sesión y emisión de firmas JWT robustas.
- Row Level Security (RLS): Capa de control de seguridad que aplica políticas directas en las tablas de PostgreSQL, validando la identidad del token emisor para prohibir accesos ilegítimos ajenos.
- Supabase Storage: Contenedor de almacenamiento en la nube encargado del

alojamiento e indexación pública de avatares fotográficos de socios y personal.

2.2.2 Justificación

La elección de Supabase descarta por completo el sobrecoste de programar, desplegar y monitorizar una API REST tradicional (ej. Java Spring Boot o Node.js Express). Al apoyarse directamente sobre un motor nativo PostgreSQL, permite mover la lógica transaccional crítica del negocio al propio motor de datos mediante funciones integradas, garantizando una comunicación segura en milisegundos mediante conexiones HTTPS protegidas por TLS.

2.2.3 Entorno de desarrollo

- Consola Web de Supabase: Utilizada para el modelado visual de tablas, configuración de cubos de almacenamiento y depuración directa de registros.
- Editor SQL de Supabase: Entorno de codificación para el despliegue de disparadores (triggers) y funciones procedurales del lado del servidor (RPC).
- Vercel: Entorno cloud de despliegue continuo automatizado para el frontend, configurado con variables de entorno blindadas.

3. Documentación Técnica

3.1 Análisis

3.1.1 Roles principales

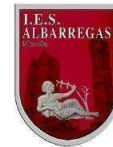
- Socio: Actor final del centro deportivo. Puede gestionar su información, reservar plazas en actividades lúdicas o de fuerza, examinar el estado técnico del material y revisar su nivel de experiencia y nivel en el gimnasio. Su portal se clausura automáticamente si no se encuentra al día con la tesorería.
- Monitor: Staff técnico deportivo del club. Tiene asignado un portal para examinar sus sesiones asignadas, pasar lista digital a los asistentes a sus clases y modificar el estado operativo de las máquinas si constata un fallo técnico.
- Administrador: Máximo responsable de la plataforma. Posee permisos absolutos para dar de alta maquinaria, contratar o destituir miembros de la plantilla, estructurar el calendario horario del gimnasio e inspeccionar las gráficas reales de facturación y flujos de caja anuales.

3.1.2 Requisitos funcionales

- **Módulo 1: Seguridad, Autenticación y Sesiones**
 - RF-01 | Registro Público de Socios: El sistema debe permitir a nuevos clientes crear una cuenta proporcionando sus datos personales y credenciales de acceso.
 - RF-02 | Inicio de Sesión (Login): El sistema debe autenticar a los usuarios mediante correo y contraseña cifrada, generando un token de sesión seguro.
 - RF-03 | Cierre de Sesión (Logout): El usuario debe poder destruir su sesión activa de forma segura para evitar accesos no autorizados en dispositivos compartidos. 3
Título del proyecto: “FITBOX”
 - RF-04 | Asignación Segura de Roles: El sistema asignará automáticamente el rol base ("Socio") a los registros públicos, requiriendo privilegios de Administrador para otorgar roles superiores (Monitor).
 - RF-05 | Control de Acceso (RBAC): La aplicación debe restringir la visualización de menús, pantallas y botones dependiendo del nivel de privilegios del usuario autenticado (Socio, Monitor o Administrador).
- **Módulo 2: Gestión del Directorio (Administración)**
 - RF-06 | Listado Global de Usuarios: El sistema debe mostrar al Administrador una tabla con el registro completo de todos los usuarios, su información de contacto y su estado.
 - RF-07 | Alta Interna de Staff: El Administrador debe disponer de un formulario privado para registrar nuevos empleados (Monitores o Co-Administradores) y darles acceso al sistema.
 - RF-08 | Modificación de Fichas: El Administrador debe poder editar los datos personales, de contacto y el rol de cualquier usuario registrado en el sistema.

Proyecto “Desarrollo de Aplicaciones Web”

Título del proyecto: “FITBOX”



- RF-09 | Baja de Usuarios: El sistema debe permitir la eliminación (baja) de perfiles de la base de datos, revocando inmediatamente su acceso a la plataforma.

- Módulo 3: Planificación y Horarios

- RF-10 | Calendario de Actividades: La plataforma debe mostrar una vista organizada con las clases programadas, indicando disciplina, monitor asignado y horario.

- RF-11 | Creación de Sesiones: El Administrador debe poder programar nuevas clases, estableciendo el aforo máximo permitido por la sala.

- RF-12 | Listado de Asistencia (Staff): Los monitores deben poder visualizar la lista de socios inscritos en las clases que ellos imparten. (Nota técnica: Esta acción dispara la asignación automatizada del sistema de gamificación).

- Módulo 4: Motor de Reservas (Socios)

- RF-13 | Inscripción en Clases: Los socios deben poder reservar plaza en las clases disponibles desde su panel de usuario. 4 Título del proyecto: “FITBOX”

- RF-14 | Cancelación de Reservas: El sistema debe permitir a los socios anular su asistencia a una clase previamente reservada, liberando la plaza.

- RF-15 | Control Automático de Aforo: El sistema bloqueará nuevas inscripciones en una clase de forma automática cuando el número de reservas iguale al aforo máximo establecido.

- Módulo 5: Gestión de Instalaciones e Inventario

- RF-16 | Catálogo de Maquinaria: El sistema debe mantener un inventario de las máquinas disponibles en las instalaciones deportivas.

- RF-17 | Reporte de Incidencias: El personal (Monitores y Administradores) debe poder alterar el estado operativo de una máquina (Ej: "Defectuoso") y añadir observaciones técnicas.
- Módulo 6: Analítica y Dashboard

- RF-18 | Panel de Métricas en Tiempo Real: El sistema debe generar un Dashboard al

iniciar sesión que resuma el estado diario del centro (número total de socios, clases activas hoy e incidencias de maquinaria pendientes).

- **Módulo 7: Mejoras Finales Implementadas (Extras del Proyecto)**

- RF-19 | Pasarela de Pagos y Control de Morosidad: Módulo de simulación de pagos con tarjeta de crédito para el abono de cuotas. El sistema bloquea el acceso a reservas a los socios con estado "Moroso".

- RF-20 | Sistema de Gamificación (XP y Niveles): Implementación de base de datos reactiva que premia con +50 XP la asistencia a clases y penaliza con -10 XP las faltas, recalculando el nivel del socio en tiempo real.

- RF-21 | Creador de Rutinas: Herramienta para que los Monitores diseñen y publiquen rutinas de entrenamiento estructuradas por días de la semana, accesibles para los socios.

3.1.3 Requisitos no funcionales

RNF- 1. Arquitectura: Single Page Application (SPA) y Patrón Repositorio

- **Descripción:** La aplicación está construida como una SPA usando React 19, TypeScript y Vite. La lógica de conexión a la base de datos se ha separado por completo de las vistas visuales mediante el patrón de diseño Repository (ej. AuthRepository, ClassRepository). El estado global (sesión y perfil) se gestiona de forma centralizada con Zustand. El backend (base de datos, almacenamiento y autenticación) se delega a Supabase (BaaS).

- **Resultado esperado:** El código es altamente modular, testeable y mantenible. La aplicación es ligera y fácil de desplegar en servicios de hosting en la nube como Vercel o Netlify.

RNF- 2. Seguridad: Autenticación, Control de Acceso (RBAC) y RLS

- **Descripción:** Se utiliza Supabase Auth para gestionar el inicio de sesión con

contraseñas cifradas (nunca en texto plano). El acceso al frontend está protegido mediante Route Guards que filtran a los usuarios según su rol (Socio, Monitor o Administrador). A nivel de backend, las tablas de PostgreSQL cuentan con políticas de seguridad Row Level Security (RLS) y procedimientos almacenados con SECURITY DEFINER.

- Resultado esperado: Un entorno blindado donde cada usuario solo accede a la información permitida por su rol, impidiendo la manipulación de datos sensibles financieros o de gamificación por terceros.

RNF- 3. Usabilidad y Accesibilidad

- Descripción: Interfaz diseñada bajo el paradigma Mobile First utilizando el framework Tailwind CSS v4, lo que garantiza una visualización perfecta en cualquier dispositivo. El código ha sido auditado mediante herramientas de análisis estático avanzado (React Doctor y ESLint) para asegurar el cumplimiento de estándares de accesibilidad. 6 Título del proyecto: “FITBOX”

- Resultado esperado: Una plataforma inclusiva, moderna e intuitiva, que proporciona una experiencia de usuario (UX) óptima tanto para clientes como para empleados del centro.

RNF- 4. Reactividad: Sincronización en Tiempo Real

- Descripción: FITBOX integra la tecnología WebSockets a través de los canales Realtime de Supabase para módulos operativos críticos. Cualquier cambio realizado por el staff en el estado de una máquina (avería) o cualquier nueva reserva de plaza en una clase, emite un evento de transmisión.
- Resultado esperado: Todos los usuarios conectados visualizan las actualizaciones de plazas, inventario y aforos instantáneamente sin necesidad de refrescar manualmente la página web.

RNF- 5. Privacidad y Protección de Datos

- Descripción: La aplicación respeta el ciclo de vida de los datos personales. Se

proporciona una herramienta administrativa para la baja y eliminación en cascada de la ficha de un usuario. Asimismo, la base de datos enmascara por defecto identificadores sensibles y la información financiera del pago simulado nunca se almacena en las tablas relacionales.

- Resultado esperado: Se cumple con los principios de protección de datos (RGPD), garantizando al socio la privacidad absoluta de su información personal y física.

3.2 Desarrollo

3.2.1 Base de Datos

La base de datos PostgreSQL de FITBOX está normalizada y estructurada para garantizar el rendimiento y la integridad de las claves foráneas. Al delegar en Supabase Auth, la tabla maestra de identidades es auth.users. A partir de ella, se extiende el esquema público mediante la tabla usuarios, que comparte el mismo UUID como clave primaria y foránea, protegiendo los privilegios de roles y los estados de suscripción.

Arquitectura del Esquema

El modelo de datos se divide en cuatro bloques lógicos para facilitar el mantenimiento y la escalabilidad:

1. Bloque de Gestión de Identidad y Roles:

- usuarios: Almacena la información perfilada del socio o empleado. Se vincula con auth.users y normaliza los permisos mediante la tabla roles.
- roles: Define los niveles de acceso (Administrador, Monitor, Socio), facilitando la escalabilidad si se decidiera añadir nuevos tipos de usuarios en el futuro.

2. Bloque de Operaciones y Reservas:

- clases y disciplinas: Estructura jerárquica que permite gestionar el catálogo de actividades y sus horarios.
- reservas: Actúa como tabla intermedia (pivote) entre clases y usuarios, resolviendo la cardinalidad N:M y garantizando la integridad referencial.

3. Bloque de Inventario y Entrenamiento:

- maquinas: Gestiona el ciclo de vida del equipamiento, incluyendo estados de avería y reportes de monitores.
- rutinas: Catálogo de planes de entrenamiento asociados a disciplinas, centralizando el contenido técnico.

4. Bloque de Gestión Financiera:

- pagos: Registro histórico de transacciones, concepto y estado (activo, pendiente), esencial para el sistema de bloqueo automático de accesos.

Integridad y Seguridad (Row Level Security - RLS)

La seguridad es el pilar de FITBOX. Se han implementado políticas RLS sobre el esquema público, garantizando que:

- Aislamiento de datos: Cada usuario solo tiene permisos de lectura sobre sus propios registros (SELECT con filtro `auth.uid() = id_socio`).
- Jerarquía de privilegios: Los roles Administrador y Monitor poseen permisos UPDATE y DELETE sobre las tablas operativas (maquinas, clases), mientras que los socios tienen acceso restringido según su estado de pago.
- Integridad referencial: Se han configurado restricciones de clave foránea (ON DELETE CASCADE en reservas, por ejemplo) para asegurar que no existan datos huérfanos tras la cancelación de un usuario o una clase.

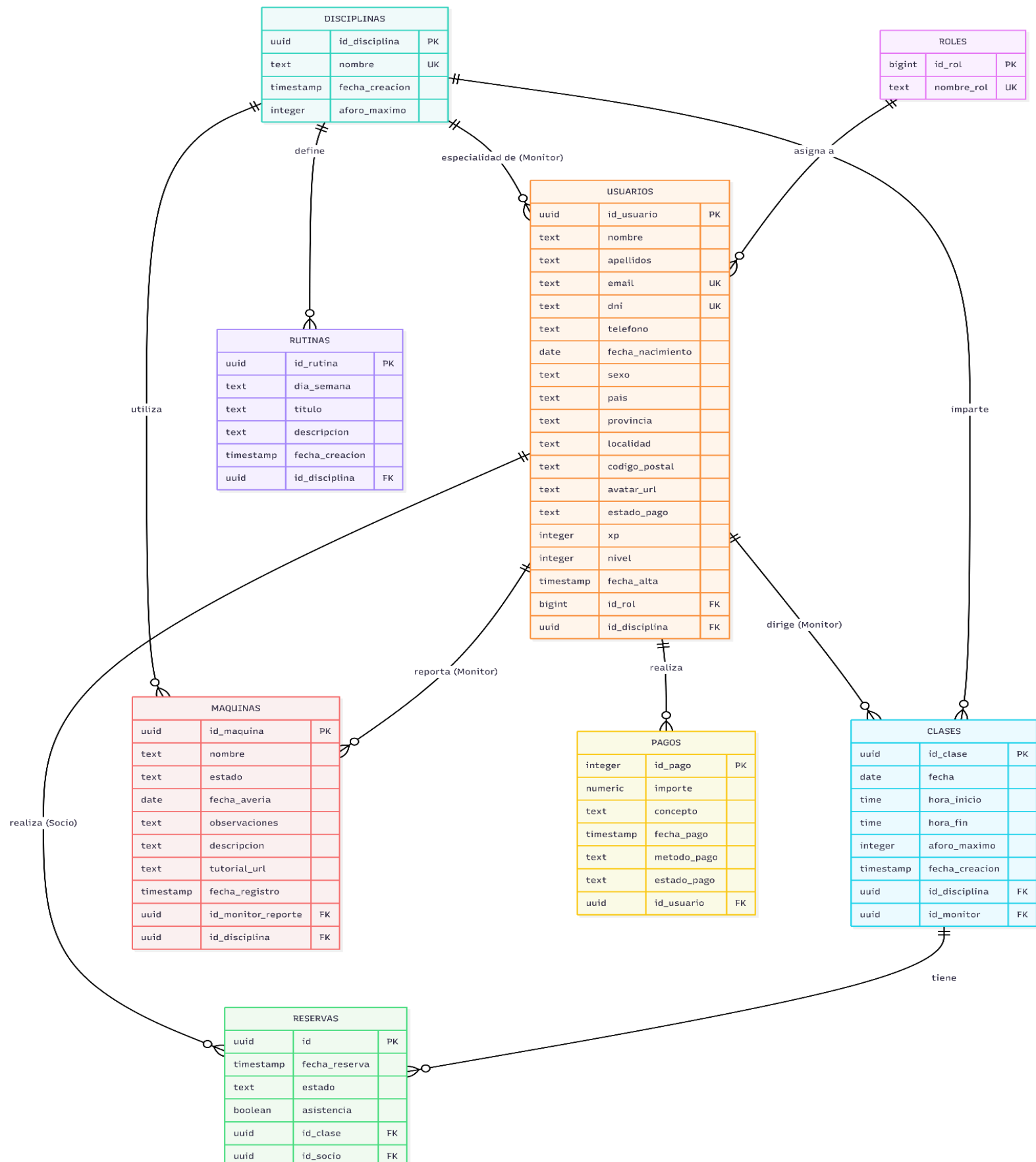
Proyecto “Desarrollo de Aplicaciones Web”

Título del proyecto: “FITBOX”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo



3.2.2 Justificación del diseño

El uso de la tabla del sistema `auth.users` como la raíz transaccional de identidades ofrece ventajas técnicas insustituibles en el desarrollo web moderno:

- Centralización Absoluta: Garantiza un único origen de la verdad para la gestión de las credenciales y el estado de autenticación del atleta.
- Políticas RLS Simplificadas: Permite blindar las tablas con sentencias de comparación directa como `auth.uid() = id_usuario`, prohibiendo de raíz que peticiones alteradas desde clientes externos puedan manipular registros ajenos.
- Lógica Procedural Robusta (Gamificación): Toda la mecánica de niveles de FITBOX está programada del lado del servidor en PostgreSQL a través de una función RPC llamada `dar_experiencia`. Al pulsar el pase de lista, el servidor inyecta los puntos (+50 XP por asistencia o -10 XP por falta) y un disparador transaccional evalúa si el acumulado sobrepasa el límite matemático del nivel ($\text{Nivel} * 200$). Al centralizar este flujo en el motor de la base de datos, se eliminan los hackeos del lado del cliente.

3.2.3 Diagrama de casos de uso

El comportamiento y los flujos operativos implementados de forma real en el código de la aplicación se detallan en el siguiente diagrama relacional de casos de uso:

Proyecto “Desarrollo de Aplicaciones Web”

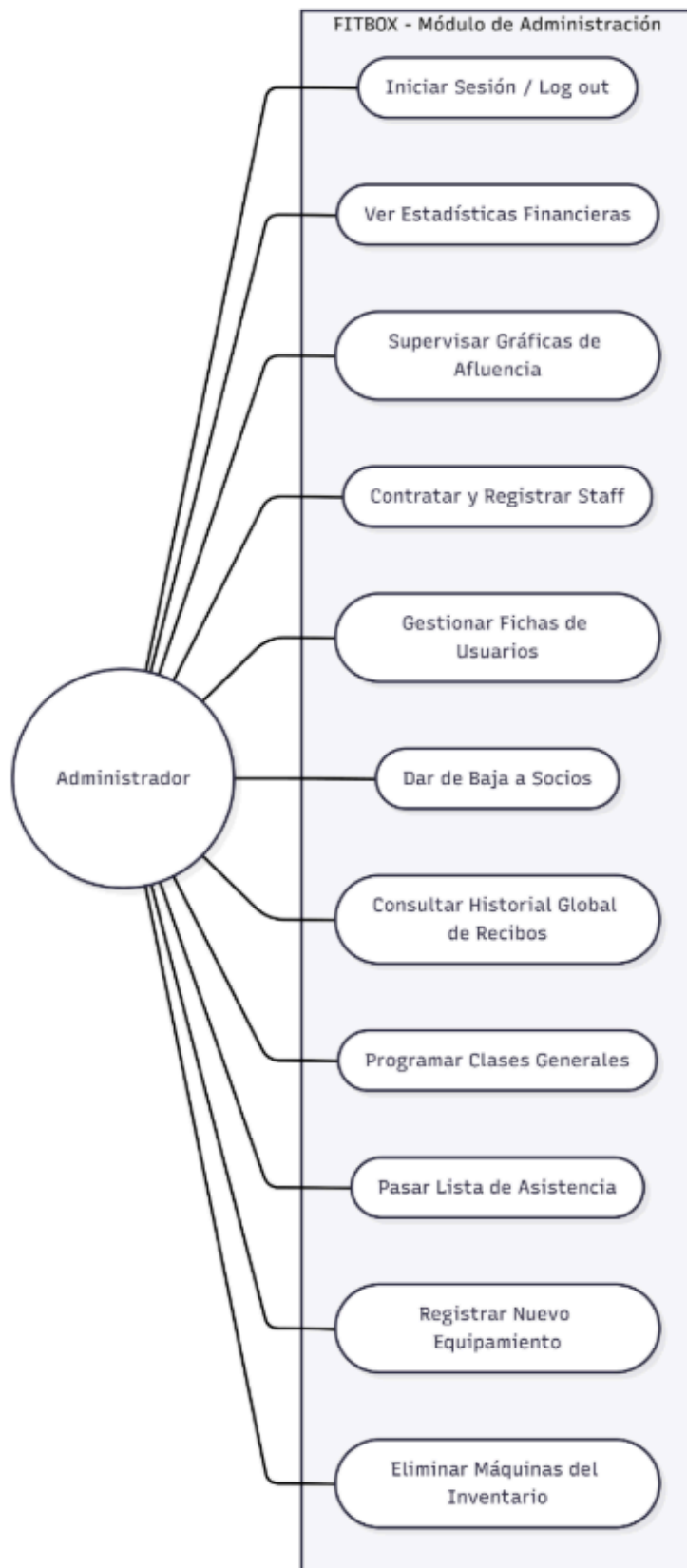
Título del proyecto: “FITBOX”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Casos de usos - Administrador



Proyecto “Desarrollo de Aplicaciones Web”

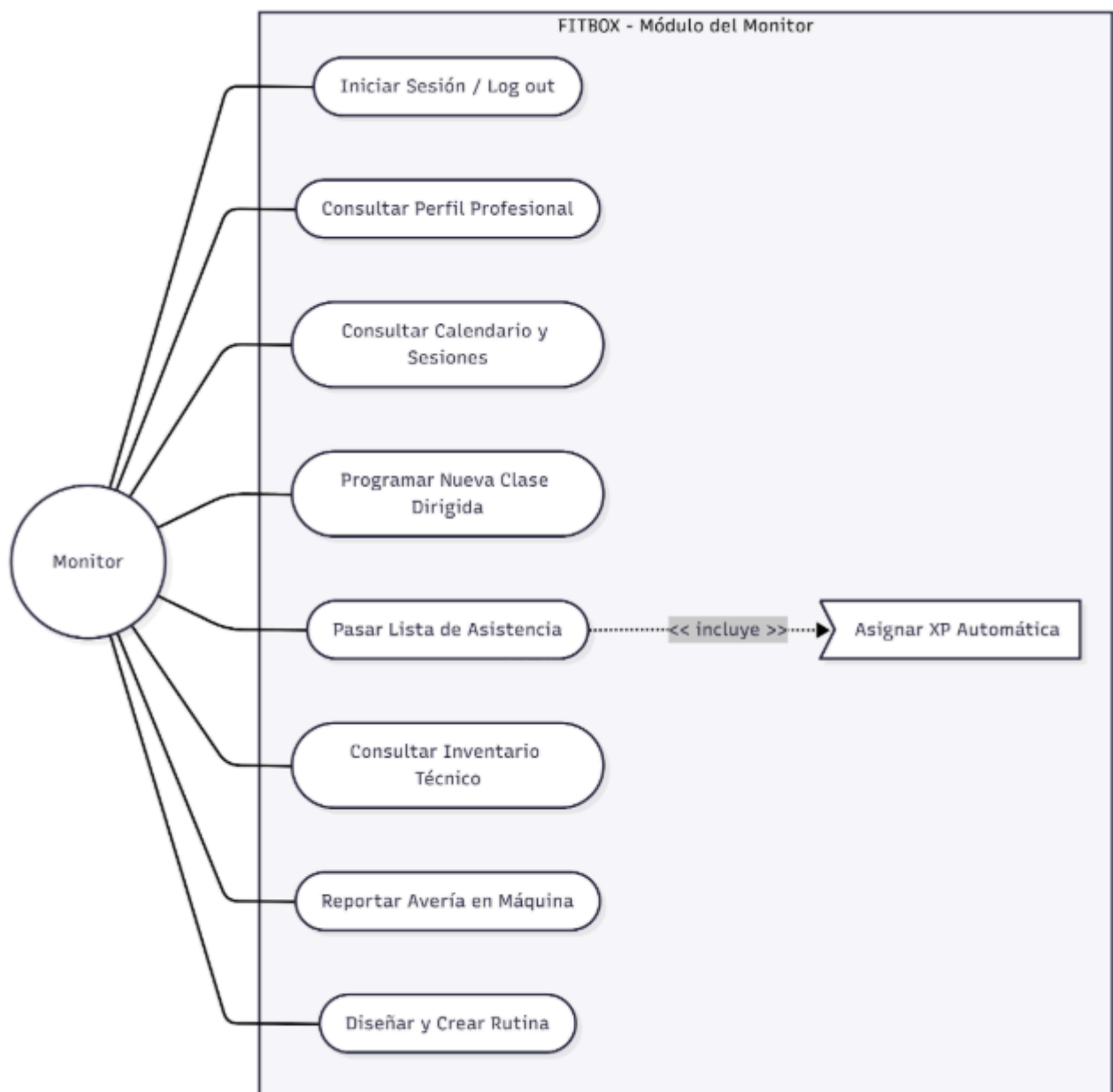
Título del proyecto: “FITBOX”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Casos de usos - Monitor



Proyecto “Desarrollo de Aplicaciones Web”

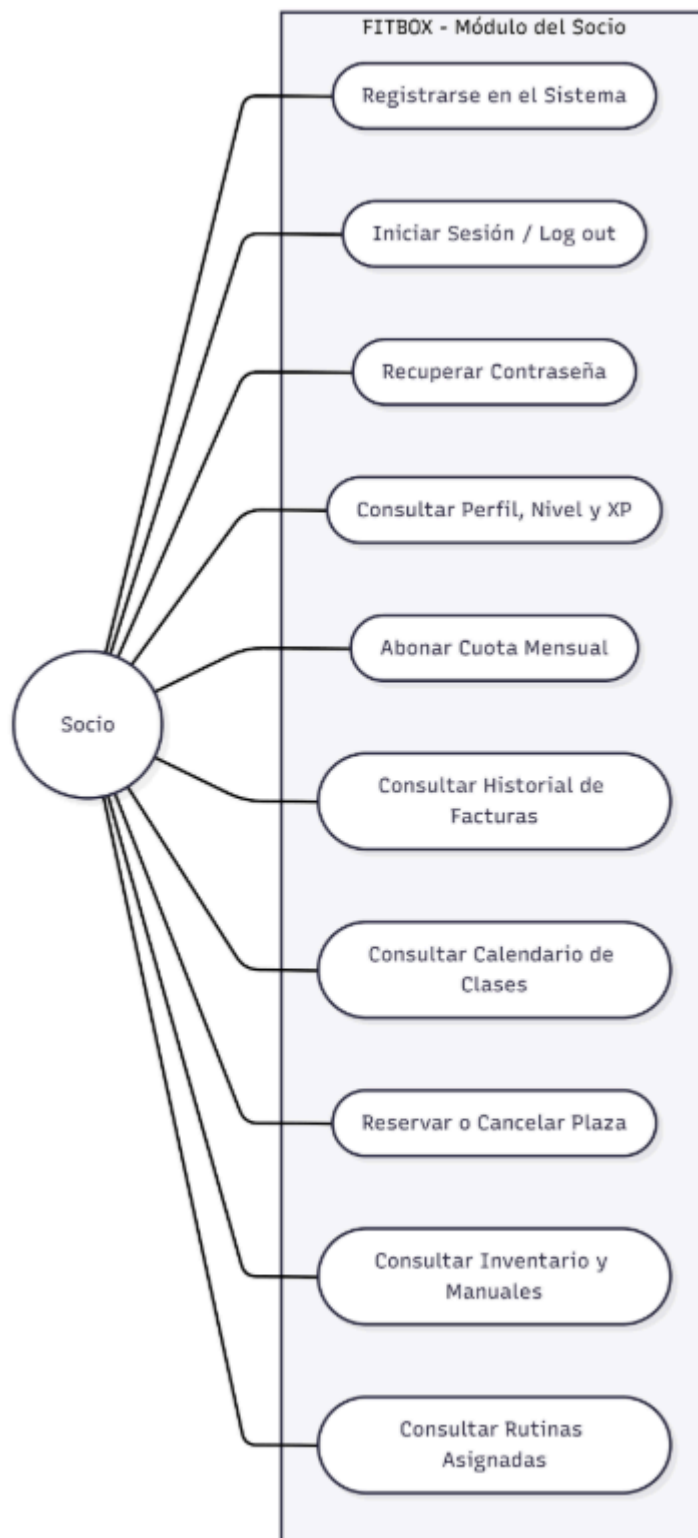
Título del proyecto: “FITBOX”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Casos de usos - Socio



3.3 Pruebas realizadas

A falta de herramientas automatizadas, se ha diseñado y ejecutado un plan sistemático de pruebas manuales de caja negra sobre la lógica crítica del negocio para certificar la estabilidad de la plataforma al 200%:

- Prueba de validación de Registro: Se testea el formulario introduciendo intencionadamente formatos inválidos en los campos. El sistema intercepta cada campo erróneo por separado y renderiza avisos específicos debajo de cada input de forma simultánea. El botón de envío permanece deshabilitado hasta que se activa la marca del checkbox de términos legales.
- Prueba de Pasarela y Máscara Financiera: Se comprueba el formulario de tarjeta de crédito en la zona de pagos. El formateador visual de React Hook Form inyecta los espacios de manera dinámica cada cuatro números. La expresión regular estricta rechaza cualquier numeración que no se corresponda con la de pruebas oficiales (4242 4242 4242 4242) o cuya fecha de vencimiento sea anterior al mes y año actuales, impidiendo la reactivación fraudulenta del perfil.
- Prueba de Bloqueo por Suscripción Inactiva: Se simula la revocación de cuota de un usuario desde el panel del Administrador o mediante el botón de baja voluntaria en el Perfil. El sistema actualiza de inmediato el registro en Supabase. Al intentar navegar a cualquier ruta operativa (ej. /clases), el guardián de rutas de React Router detecta que el estado de pago no es activo y bloquea la interfaz por completo mediante una pantalla de acceso restringido.
- Prueba de Pase de Lista y Triggers de Experiencia (XP): Desde el perfil de Monitor, se accede al listado de socios apuntados a una clase y se presiona el botón verde de asistencia. La base de datos registra la acción, dispara la función almacenada y actualiza el nivel de experiencia del socio en tiempo real mediante WebSockets, viéndose reflejado en el perfil del socio de inmediato.

4. Proceso de despliegue

El ecosistema digital de FITBOX se encuentra en un entorno de producción real y operativo en la nube, dividido en dos infraestructuras independientes de alta disponibilidad (Supabase para los servicios de datos y Vercel para el alojamiento del código cliente).

4.1 Software necesario

- Navegador web moderno (Google Chrome o Firefox recomendado).
- Cuenta de usuario en GitHub (Repositorio privado).
- Cuenta de desarrollador en Supabase.
- Cuenta de despliegue en Vercel.
- Node.js y npm instalados en el equipo local.

4.2 Pasos para desplegar la aplicación

4.2.1 Configurar Supabase

1. Crear un nuevo proyecto en la plataforma de Supabase.
2. Ejecutar los scripts SQL necesarios para generar la estructura de tablas (usuarios, pagos, clases, reservas, maquinas, rutinas).
3. Implementar las políticas de Row Level Security (RLS) que garanticen que cada socio solo acceda a su información privada.
4. Activar el servicio de Supabase Auth para gestionar el ciclo de vida de los usuarios mediante email.
5. Extraer los identificadores VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY desde la configuración del proyecto.

Proyecto “Desarrollo de Aplicaciones Web”

Título del proyecto: “FITBOX”



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

4.2.2 Configurar el proyecto local

6. Clonar el repositorio privado desde GitHub:

```
git clone https://github.com/agarciai03/FITBOX.git
```

```
cd FITBOX
```

7. Crear un archivo .env en la raíz (asegurando que esté en el .gitignore) con las credenciales obtenidas:

```
VITE_SUPABASE_URL=....
```

```
VITE_SUPABASE_ANON_KEY=....
```

8. Instalar dependencias y ejecutar en modo desarrollo:

```
npm install
```

```
npm run dev
```

4.2.3 Despliegue en Vercel

9. Iniciar sesión en Vercel y seleccionar "Import Project" desde GitHub.

10. Vincular el repositorio privado de FITBOX.

11. En la configuración, añadir las variables de entorno (VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY) exactamente con los mismos valores definidos en local.

12. Lanzar el despliegue.

13. Una vez finalizado, Vercel genera una URL pública con certificado SSL (HTTPS) que servirá de acceso a toda la comunidad de usuarios. Cualquier push a la rama main activará un nuevo despliegue automático.

5. Propuestas de mejoras

Durante el desarrollo se han detectado oportunidades para escalar FITBOX:

- Pasarela de Pago Real: Sustituir el simulador actual por una implementación oficial de Stripe con Webhooks para gestionar suscripciones mensuales reales.
- Notificaciones Push: Implementar Firebase Cloud Messaging para avisar a los socios en tiempo real si una plaza en lista de espera queda libre.
- Gestión de Lista de Espera: Automatizar mediante Triggers la asignación de plazas vacantes al primer usuario en espera cuando otro socio cancele su reserva.
- Reportes PDF Avanzados: Incluir la exportación de facturas detalladas y reportes de progreso físico histórico para el socio.
- Personalización de emails: Adaptar las plantillas de correo de Supabase para incluir el logo y los colores corporativos de FITBOX.

6. Bibliografía

- React: Documentación oficial sobre Hooks y componentes funcionales.<https://es.react.dev/>
- Vite: Guía para optimización de builds en React.<https://es.vite.dev/guide/>
- Tailwind CSS: Documentación técnica sobre clases utilitarias y diseño responsivo.
<https://tailwindcss.com/>
- Supabase: Referencia completa de base de datos, Auth y RLS.
<https://supabase.com/docs>
- Vercel: Despliegue continuo de aplicaciones front-end.<https://vercel.com/>
- Zustand: Documentación para la gestión de estados globales.<https://docs.pmnd.rs/zustand/>